

Pydantic and MCP: Why Both Are Needed in Agentic AI Pipelines

A Defence-in-Depth Analysis with Examples from PathwiseAI

PathwiseAI Project

May 2026

Contents

1	The Question	2
2	What MCP Actually Does	2
2.1	The Protocol	2
2.2	What MCP Validates	3
2.3	What MCP Does <i>Not</i> Validate	3
3	Where LLMs Break the Contract	3
4	What Pydantic Adds	4
4.1	Type Coercion	4
4.2	Field Defaults at the Server Side	4
4.3	Constraint Enforcement	5
4.4	Structured Output from Tool Results	5
5	The Architecture: Two Distinct Layers	6
6	Failure Cascade Without Pydantic: A Concrete Example	6
7	Pydantic in the PathwiseAI MCP Client	8
8	Why This Matters for Agentic AI Goals	9
8.1	Autonomous Multi-Step Execution	9
8.2	Long Context Accumulation	9
8.3	Determinism and Reproducibility	9
8.4	Tool Composability	9
8.5	Security	10
9	The FastMCP–Pydantic Integration	10
10	Summary	11

1 The Question

The **Model Context Protocol** (MCP) already handles the full round-trip between an LLM and a tool: it converts the LLM’s JSON tool-call into Python function arguments, executes the function, and converts the Python return value back into JSON for the LLM to read. Given this, a natural question arises:

“If MCP already converts $JSON \rightarrow Python$ args and $Python \rightarrow JSON$, why add Pydantic on top? Aren’t we validating the same thing twice?”

The short answer is: **no, they are not validating the same thing**. MCP operates at the *transport layer*—it guarantees that bytes arrive and are structurally well-formed JSON. Pydantic operates at the *application layer*—it guarantees that those bytes mean what your business logic needs them to mean. This document explains the distinction with concrete examples, shows where each layer fails without the other, and explains why the combination is essential for robust agentic AI systems.

2 What MCP Actually Does

2.1 The Protocol

MCP uses **JSON-RPC 2.0** as its wire format. When a Gemini or Claude model decides to call a tool, the MCP client sends a message that looks like this over `stdio` (or HTTP SSE):

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "get_top_youtube_channels",
    "arguments": {
      "query": "Reinforcement Learning",
      "max_pages": "3",
      "relevance_language": null
    }
  }
}
```

Listing 1: MCP tool-call wire message (JSON-RPC 2.0)

The MCP server library (e.g. `fastmcp`) receives this, looks up the registered tool, and calls the Python function with the `arguments` dict. The Python return value is then serialised back to JSON and sent as:

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "result": {
    "content": [
      { "type": "text", "text": "{\"channels\": [...], \"search_locale\": {...}]" }
    ]
  }
}
```

```

    ],
    "isError": false
  }
}

```

Listing 2: MCP tool response (JSON-RPC 2.0)

2.2 What MCP Validates

FastMCP generates a **JSON Schema** from each tool’s Python type annotations and exposes it to clients via `tools/list`. At call time, it validates the incoming **arguments** object against that schema *structurally*:

- Is the payload valid JSON? (syntax check)
- Are all required fields present?
- Are the JSON types nominally correct (string, number, boolean, array, object)?

2.3 What MCP Does *Not* Validate

JSON Schema has fundamental expressive limits. MCP cannot enforce:

1. **Type coercion.** JSON "3" (string) is not JSON 3 (number). If your function expects `int` but the LLM sends "3", MCP passes the string through unchanged and your code may crash or silently misbehave.
2. **Business-logic constraints.** JSON Schema can express `minimum: 1`, but it cannot enforce that a value is an element of a dynamic whitelist, or that two fields are mutually exclusive.
3. **Cross-field rules.** “If `status` is `pending` then `due_date` is required” cannot be expressed in standard JSON Schema.
4. **Default application at the application layer.** MCP uses defaults in the *schema* it advertises to clients, but those defaults are applied by the *client*, not enforced by the server. If the client omits a field the server receives nothing for it.
5. **Semantic correctness.** A YouTube channel ID like "not-a-real-id" is a valid string—JSON Schema will accept it.

3 Where LLMs Break the Contract

LLMs are probabilistic text generators. Even with strict system prompts, they produce structurally invalid or semantically wrong tool arguments with surprising frequency. OpenAI’s own benchmarks found that without runtime enforcement, GPT-4 violated its declared output schema **approximately 40%** of the time. Common failure patterns include:

JSON string wrapping (last row) is particularly subtle and well documented. Some MCP clients, including older Claude Desktop versions, serialise complex arguments as JSON strings rather than native JSON objects, so your server receives a stringified array instead

Failure Pattern	LLM sends	Tool expects
Type confusion	"max_pages": "3"	int
Out-of-range value	"top_n": -1	int > 0
Missing optional	{}	field = default
Extra whitespace	" IN "	"IN"
Wrong key name	"lang": "en"	"relevance_language"
Invented value	"region_code": "EU"	ISO 3166-1 alpha-2
JSON string wrapping	"[1,2,3]" (string)	list[int]

Table 1: Common LLM tool-call failures that MCP passes but Pydantic catches

of a real array. MCP passes it through; without Pydantic you discover this only when iterating the string character by character.

4 What Pydantic Adds

Pydantic is a **runtime data validation and coercion library**. Where MCP validates *structure*, Pydantic validates *semantics*.

4.1 Type Coercion

Pydantic’s default (“lax”) mode automatically converts compatible types:

```
from pydantic import BaseModel, Field

class ChannelQuery(BaseModel):
    query: str
    max_pages: int = 2          # LLM may send "3" (string) ->
                               # coerced to 3 (int)
    region_code: str | None = None

# LLM produced this (note the string "3")
raw = {"query": "Reinforcement Learning", "max_pages": "3"}

q = ChannelQuery.model_validate(raw)
print(q.max_pages)      # 3 (int, not "3")
print(q.region_code)    # None (default applied)
```

Listing 3: Pydantic coerces types the LLM got wrong

4.2 Field Defaults at the Server Side

MCP schemas advertise defaults so that *clients* may omit fields. But in an agentic loop the LLM is the client, and it will omit fields whenever it deems them irrelevant. Pydantic applies the default at the *server side*, making behaviour deterministic regardless of what the LLM chose to include:

```
class VideoQuery(BaseModel):
    channel_link: str
```

```

    top_n: int = Field(default=5, ge=1, le=10)

# LLM omits top_n entirely
raw = {"channel_link": "https://www.youtube.com/@3blue1brown"}

q = VideoQuery.model_validate(raw)
print(q.top_n)    # 5 -- predictable, no crash

```

Listing 4: Server-side defaults via Pydantic

4.3 Constraint Enforcement

JSON Schema can express simple numeric bounds, but Pydantic validates them *at the server* with a clear error message the agent can act on:

```

from pydantic import BaseModel, Field, field_validator
import re

class ChannelData(BaseModel):
    title: str = ""
    url: str = ""
    subscribers: int = 0          # int, not float or string
    views: int = 0
    videos: int = 0
    score: float = 0.0

    @field_validator("url")
    @classmethod
    def url_must_be_youtube(cls, v: str) -> str:
        if v and not re.match(r"https?://(www\.)?youtube\.com/",
                               v):
            raise ValueError(f"Not a YouTube URL: {v!r}")
        return v.strip()

```

Listing 5: Constraint enforcement catches out-of-range values

4.4 Structured Output from Tool Results

Pydantic is equally valuable on the *output* path: when a tool returns data that the MCP client will parse back into a dict for the LLM, validating the output with Pydantic ensures the response conforms to a known schema:

```

class ChannelOut(BaseModel):
    title: str = ""
    channel_id: str = ""
    url: str = ""
    subscribers: int = 0
    views: int = 0
    videos: int = 0
    score: float = 0.0

def _normalize_channels(channels: list[dict]) -> list[dict]:

```

```
# Each channel dict from the YouTube API is validated and
# normalised.
# Missing keys become defaults; wrong types are coerced.
return [ChannelOut.model_validate(ch).model_dump() for ch in
        channels]
```

Listing 6: Normalising server output with Pydantic (mcp_server.py)

Without this, a YouTube API response that omits `subscriberCount` for a private-subscriber channel would propagate `None` into downstream code, causing arithmetic failures when computing the relevance score.

5 The Architecture: Two Distinct Layers

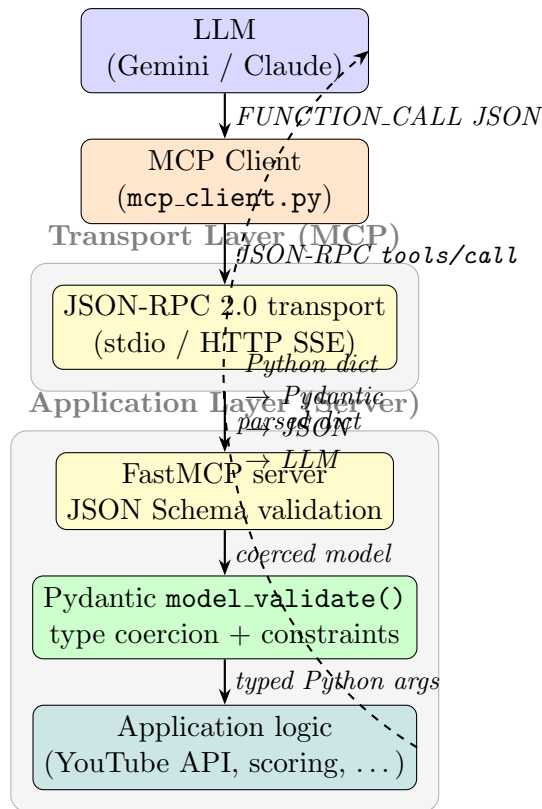


Figure 1: MCP and Pydantic occupy different layers. MCP validates transport structure; Pydantic validates application semantics.

6 Failure Cascade Without Pydantic: A Concrete Example

Consider the `get_top_video_stats` tool in PathwiseAI. Its signature is:

```
@mcp.tool()
def get_top_video_stats(channel_link: str, top_n: int = 5) ->
    dict:
        top_n = max(1, min(int(top_n), 10)) # defensive clamp
```

Concern	MCP handles it	Pydantic handles it
JSON parsing	Yes – rejects malformed JSON	No
Required-field presence	Yes – per JSON Schema	Yes – raises <code>ValidationError</code>
Type coercion	No – passes wrong types through	Yes – <code>"3" → 3</code>
Default values	Client-side only	Server-side, guaranteed
Value constraints	Limited (<code>min/max</code>)	Full (<code>ge</code> , <code>le</code> , <code>regex</code> , <code>custom</code>)
Cross-field logic	No	Yes – <code>model_validator</code>
Output normalisation	No	Yes – <code>model_dump()</code>
Error messages	Generic schema violation	Field-level, human-readable
Alias / renaming	No	Yes – <code>Field(alias=...)</code>

Table 2: Division of responsibility between MCP and Pydantic

...

Now trace what happens when the LLM (under token pressure) emits:

```
{
  "tool_name": "get_top_video_stats",
  "tool_arguments": {
    "channel_link": "https://www.youtube.com/@3blue1brown",
    "top_n": "five"
  }
}
```

Listing 7: LLM-generated tool call with two type errors

Without Pydantic (MCP layer only):

1. MCP receives `"five"` (a string). JSON Schema says `top_n` is type `integer`. MCP *may* raise a schema error, or the schema may be lenient.
2. If it passes through, `int("five")` raises a `ValueError`.
3. The exception propagates up to MCP’s error handler, which returns `{"isError": true, "content": "ValueError: invalid literal..."}`.
4. The LLM receives a raw Python traceback. It may retry correctly, or it may hallucinate a fix that introduces a new error. The agent has consumed an entire iteration with no productive output.

With Pydantic (both layers):

```
from pydantic import BaseModel, Field

class VideoStatsInput(BaseModel):
    channel_link: str
    top_n: int = Field(default=5, ge=1, le=10)
```

```

# At tool entry point:
validated = VideoStatsInput.model_validate(tool_args)
# Raises: ValidationError: 1 validation error for VideoStatsInput
#   top_n
#       Input should be a valid integer, unable to parse string as
#       an integer
#       [type=int_parsing, input_value='five', ...]

```

Listing 8: Pydantic catches the error before application code runs

The LLM receives a *structured*, field-level error message. Because Pydantic’s error identifies exactly which field is wrong and why, the LLM can immediately emit a corrected call on the next iteration—no wasted tool calls, no cascade.

7 Pydantic in the PathwiseAI MCP Client

PathwiseAI’s `mcp_client.py` has an equally important use of Pydantic on the **response** side. After the agent calls `get_top_youtube_channels`, the MCP server returns a JSON payload containing a `channels` array. The client must parse that array and write it as a canonical text file for downstream chart generation. Here the LLM is never involved—the client does it deterministically using Pydantic models:

```

class ChannelData(BaseModel):
    title: str = ""
    url: str = ""
    subscribers: int = 0
    views: int = 0
    videos: int = 0
    score: float = 0.0

def extract_channels_from_payload(payload: str) -> list[
ChannelData] | None:
    try:
        data = json.loads(payload)
        items = data.get("channels", []) if isinstance(data, dict)
            else []
        result = [ChannelData.model_validate(ch) for ch in items
                    if isinstance(ch, dict)]
        return result or None
    except Exception:
        return None

def format_channels_dump(channels: list[ChannelData]) -> str:
    blocks = [
        f"{i}. {ch.title}\nURL: {ch.url}\nSubscribers: {ch.
            subscribers}\n"
        f"Views: {ch.views}\nVideos: {ch.videos}\nScore: {ch.
            score:.4f}"
        for i, ch in enumerate(channels, 1)
    ]
    return "\n\n".join(blocks)

```

Listing 9: Client-side Pydantic: parsing tool results reliably

Without Pydantic here, the client would need:

- `int(ch.get("subscribers", 0) or 0)` — guards for `None`, string, missing key
- `float(ch.get("score", 0.0) or 0.0)` — same for floats
- `str(ch.get("title", "")).strip()` — same for strings

This is exactly what existed before the refactor (seven lines of defensive casts per field per model). Pydantic replaces all of it with `model_validate(ch)`, and makes it easier to extend: adding a new field means one line in the model, not three lines of defensive casting in every consumer.

8 Why This Matters for Agentic AI Goals

Agentic AI systems have properties that make reliable validation especially critical:

8.1 Autonomous Multi-Step Execution

An agent runs a loop of: *observe* → *reason* → *act* → *observe*. A validation failure in step k means the agent either crashes or receives a bad observation, which corrupts its reasoning for steps $k+1$ through N . A single type error in a tool argument can cascade into five wasted iterations before the agent reaches `FINAL_ANSWER`.

Pydantic catches the error at the boundary, returns a clear message, and lets the agent self-correct *within the same iteration* rather than wasting a full round-trip.

8.2 Long Context Accumulation

Each agent iteration appends to a growing prompt (the `history` list in `mcp_client.py`). Validation errors add noise to this context. With a token budget of 12 iterations (`MAX_ITERATIONS = 12`), wasting two iterations on malformed arguments is a 17% budget loss. Pydantic’s immediate, targeted errors minimise iteration waste.

8.3 Determinism and Reproducibility

A Pydantic model defines the *canonical shape* of data at a boundary. Every consumer of that boundary sees the same types and defaults, regardless of which LLM produced the input or which version of the YouTube API produced the output. This makes agent behaviour reproducible— a prerequisite for debugging, testing, and trustworthy deployment.

8.4 Tool Composability

In a multi-tool pipeline, the output of one tool becomes the input of the next. Pydantic models act as **contracts** between tools. When `get_top_youtube_channels` returns a `list[ChannelData]`, any downstream tool that accepts `ChannelData` is guaranteed to

receive well-typed data. This is the same reason function signatures in statically typed languages enable safe refactoring—Pydantic brings the same guarantee to the dynamic runtime of an agentic loop.

8.5 Security

MCP’s sandboxed file tool (`write_file`, `read_file`) uses path validation to prevent directory traversal. Pydantic adds a complementary layer: field validators can reject paths containing shell metacharacters, enforce maximum content lengths, or whitelist allowed file extensions—concerns that are outside MCP’s protocol scope but inside the application’s security perimeter.

9 The FastMCP–Pydantic Integration

FastMCP has its own Pydantic integration, which is worth distinguishing from user-defined models. FastMCP uses Pydantic internally to:

1. Generate JSON Schema from Python type annotations for `tools/list`.
2. Validate and coerce the `arguments` dict into the function’s parameter types before calling the handler (default: lax/flexible mode).

This is helpful, but it operates only on *tool parameters* declared in the function signature. It does *not* validate:

- The internal structure of complex `dict` parameters (a `dict` parameter passes FastMCP validation even if required nested keys are absent).
- The return value of the function.
- Data that flows *through* a tool from an external API (like the YouTube API returning `null` for `subscriberCount`).

User-defined Pydantic models fill these gaps. They give the developer explicit control over validation at the boundaries that FastMCP does not see.

```
# FastMCP coerces this automatically (good):
@mcp.tool()
def get_top_video_stats(channel_link: str, top_n: int = 5) -> dict:
    ...

# FastMCP does NOT validate nested dict content (gap):
@mcp.tool()
def process_channel(channel: dict) -> dict:
    # 'channel' arrives as a raw dict -- no guarantee about its
    # keys/types
    subs = int(channel["subscribers"])    # KeyError or TypeError
    if LLM was sloppy
    ...

# Pydantic closes the gap:
@mcp.tool()
```

```
def process_channel(channel: dict) -> dict:
    ch = ChannelData.model_validate(channel)  # guaranteed keys,
        coerced types
    subs = ch.subscribers                    # always int,
        always present
    ...
```

Listing 10: FastMCP’s built-in coercion vs. explicit Pydantic model

10 Summary

MCP and Pydantic are complementary, not competing. They solve different problems at different layers of an agentic pipeline:

MCP solves the *connectivity problem*: how does an LLM discover, invoke, and receive results from tools in a standardised, multi-vendor ecosystem?

Pydantic solves the *correctness problem*: how do we guarantee that the data crossing each boundary has the right types, the right values, and the right shape for our application logic—regardless of whether the producer was an LLM, an external API, or another tool?

An agentic system without MCP is not interoperable. An agentic system without Pydantic is fragile. The combination produces systems that are both interoperable *and* robust—the two non-negotiable properties of production-grade agentic AI.

References

- [1] Model Context Protocol Specification (2025-06-18) — Tools. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- [2] FastMCP Documentation — Tools. <https://gofastmcp.com/servers/tools>
- [3] Pydantic Documentation — `model_validate`. <https://docs.pydantic.dev/latest/>
- [4] Pydantic AI Documentation. <https://ai.pydantic.dev/>
- [5] Finndersen, “How to Use MCP Tools with a PydanticAI Agent”, Medium, 2025.
- [6] Galileo, “7 AI Agent Failure Modes and How to Prevent Them”, 2025. <https://galileo.ai/blog/agent-failure-modes-guide>
- [7] Lanham, “Stop Blaming the LLM: JSON Schema Is the Cheapest Fix for Flaky AI Agents”, Medium, 2025.
- [8] DasRoot, “Model Context Protocol: A Technical Deep Dive”, 2026. <https://dasroot.net/posts/2026/04/model-context-protocol-mcp-technical-deep-dive/>
- [9] Merge.dev, “MCP Tool Schema: What It Is, How It Works, and Examples”, 2025. <https://www.merge.dev/blog/mcp-tool-schema>
- [10] FastMCP GitHub Issue #932 — “JSON arguments as string”. <https://github.com/jlowin/fastmcp/issues/932>